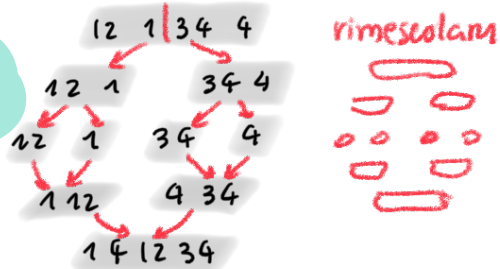


ORDINAMENTO

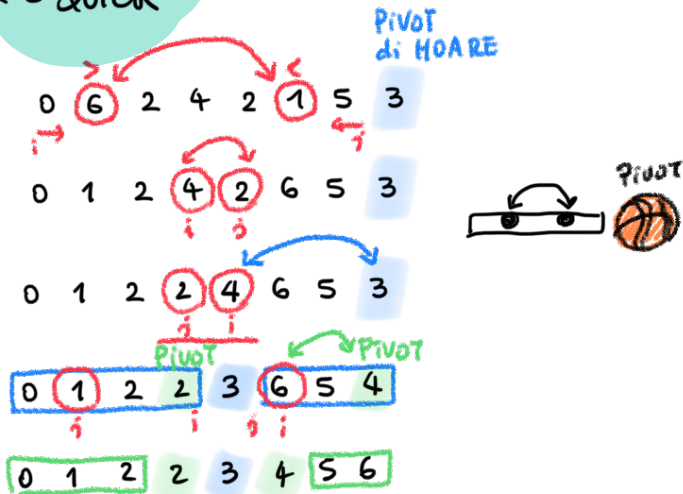
Ricorsivi

• MERGE



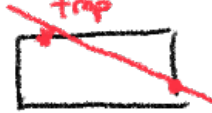
rimescolare

• Quick



NON Ricorsivi

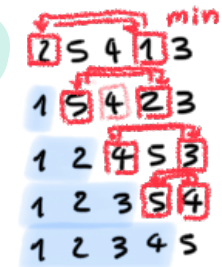
• INSERTION



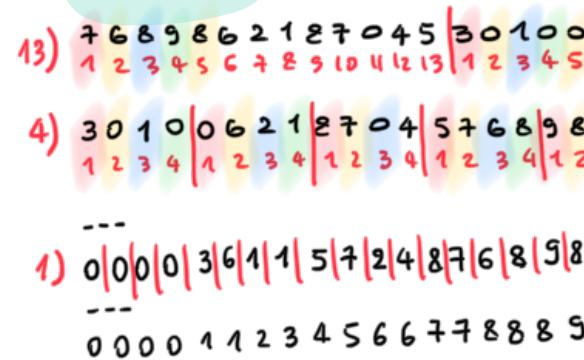
• BUBBLE



• SELECTION



• SHELL

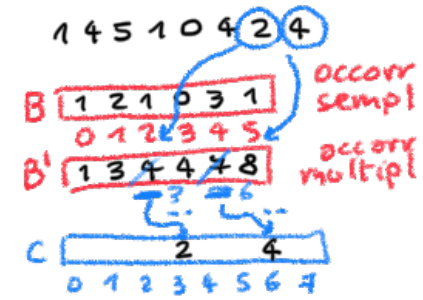


1 4 13 ...

insertion sort
ai Solovvdi di
n° distanti li

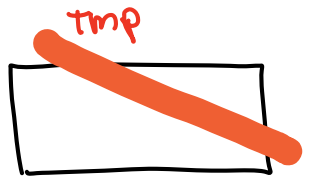
• COUNTING

vett. init. A
vett. occ. B
vett. ord. C

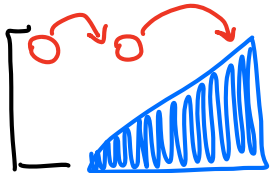




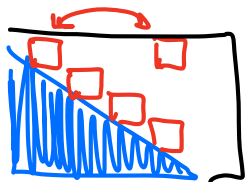
ORDINAMENTO



INSERTION



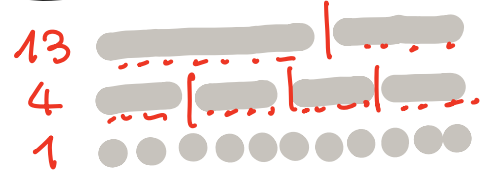
BUBBLE



SELECTION

NON
RICORSIVI

SHELL



COUNTING



Sviluppo

COND. di TERM

$$T(n) = 3T\left(\frac{n}{2}\right) + \frac{n}{2}$$

$$T(1) = 1$$

$$T\left(\frac{n}{2}\right) = 3T\left(\frac{n}{4}\right) + \frac{n}{4}$$

$$T\left(\frac{n}{4}\right) = 3T\left(\frac{n}{8}\right) + \frac{n}{8}$$

$$\frac{n}{2^i} = 1$$

$$i = \log_2 n$$

Sostit

$$\begin{aligned} T(n) &= 3 \left[3 \left[3T\left(\frac{n}{8}\right) + \frac{n}{8} \right] + \frac{n}{4} \right] + \frac{n}{2} = \\ &= 27T\left(\frac{n}{8}\right) + 9\frac{n}{8} + 3\frac{n}{4} + \frac{n}{2} = \\ &= \frac{n}{2} \left[1 + \frac{3}{2} + \frac{9}{4} + \dots \right] = \\ &= \frac{n}{2} \sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i \end{aligned}$$

PROGRESSIONE GEOMETRICA

$$\sum_{i=0}^n x^i = \frac{x^{n+1} - 1}{x - 1}$$

$$\begin{aligned} T(n) &= \frac{n}{2} \left[\frac{\left(\frac{3}{2}\right)^{\log_2 n + 1} - 1}{\frac{3}{2} - 1} \right] \approx \frac{n}{2} \left[\left(\frac{3}{2}\right)^{\log_2 n} - 1 \right] = \\ &= \frac{n}{2} \left[n^{\log_2 \frac{3}{2}} - 1 \right] = \frac{n}{2} \left[n^{\log_2 3 - \log_2 2} - 1 \right] = \\ &= \frac{n}{2} \left[\frac{1}{n} \cdot n^{\log_2 3} - 1 \right] = O(n^{\log_2 3}) \end{aligned}$$

Sol

OB

$O(\dots)$ di
ALGORITMI
RICORSIVI

EQUAZIONI alle RICORRENTE

PROGR. GEOM.

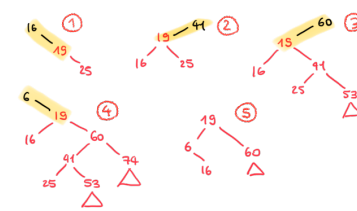
$$\sum_{i=0}^n x^i = \frac{x^{n+1} - 1}{x - 1}$$

PROGR. ARITMETICA

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

$$a^{\log_2 b} = b^{\log_2 a}$$

$$a^{\log_2 \frac{b}{c}} = a^{\log_2 b - \log_2 c} = \frac{a^{\log_2 b}}{a^{\log_2 c}}$$

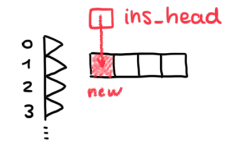


A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10
K	L	M	N	O	P	Q	R	S	T
11	12	13	14	15	16	17	18	19	20
U	V	W	X	Y	Z				
21	22	23	24	25	26				

HASH TABLE

LINEAR CHAINING

$$h(k) = k \bmod \dots$$



1 8 5 12 1 0 1 0 1 1 19 4
T₁ H E L A₁ S₁ T₂ S₂ T₃ A₂ N D
 $h(k) = k \bmod 19$

0	S ₂ S ₁	10
1	A ₂ T ₃ T ₂ A ₁ T ₁	11
2		12
3		13
4	D	14
5	E	15
6		16
7		17
8	H	18
9		19

OPEN ADDRESSING

LINEAR PROBING

$$h(k) = k \bmod \dots$$



10 14 6 1 0 8 0
27 14 23 1 17 8 85
1 6 11 2 8
52 6 11 19 25

$$M=17 \rightarrow h(k) = k \bmod 17$$

0	17	10	27
1	1	11	11
2	85	12	
3	52	13	
4	19	14	14
5		15	
6	23	16	
7	6		
8	8		
9	25		

QUADRATIC PROBING

$$\text{ind} = \begin{cases} h(k) \\ h(k) + \underline{1} + \underline{1}^2 \end{cases}$$

COEFFICIENTI: $C_1 = C_2 = 1$

$T = n^{\circ}$ di tentativi 1, 2, 3, ...

4 5 18 13 1 20 15 3 12 15
D E R M A T O G L Y
16 8 9 3 19
P H I C S

$$M=19 \rightarrow h(k) = k \bmod 19$$

$C_1 = C_2 = 1$

0	S	10	L
1	A	11	H
2	T	12	C
3	D	13	O
4	E	14	P
5	R	15	
6		16	
7		17	
8		18	
9			

$$T = 1 + 1 + 1^2 = 3$$

$$C = 3 + 1 + 1^2 = 5$$

$$C = 3 + 2 + 2^2 = 9$$

$$C = 3 + 3 + 3^2 = 18$$

$$C = 3 + 4 + 4^2 = 23 \times 19 = 4$$

$$C = 3 + 5 + 5^2 = 33 \times 19 = 14$$

DOUBLE HASHING

$$\text{index} = \begin{cases} h_1(k) \\ h_1(k) + \underline{1} \cdot h_2(k) \end{cases}$$

$$h_2(k) = 1 + k \times 97$$

$T = n^{\circ}$ di tentativi 1, 2, 3, ...

9 4 32 11 19 8 51 28 38
37 43

$$M=17 \rightarrow h_1(k) = k \bmod 17$$

$$h_2(k) = 1 + k \times 97$$

0	51	10	11
1	19	11	43
2	37	12	
3	4	13	
4		14	38
5	28	15	32
6	8	16	
7	5		
8			
9			

'28'

$$h_1 = 11$$

$$h_2 = 1 + 28 \times 97 = 29$$

$$\rightarrow (h_1 + 1 \cdot h_2) \times 17 = 6$$

'38'

$$h_1 = 4$$

$$h_2 = 1 + 38 \times 97 = 39$$

$$\rightarrow (h_1 + 1 \cdot h_2) \times 17 = 9$$

$$\rightarrow (h_1 + 2 \cdot h_2) \times 17 = 14$$

'43'

$$h_1 = 5$$

$$h_2 = 1 + 43 \times 97 = 44$$

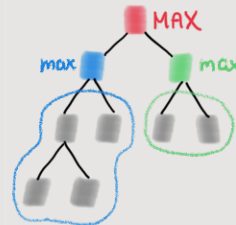
$$\rightarrow (h_1 + 1 \cdot h_2) \times 17 = 2$$

$$\rightarrow (h_1 + 2 \cdot h_2) \times 17 = 12$$

HEAP

implementa

COMPLETE BINARY TREE



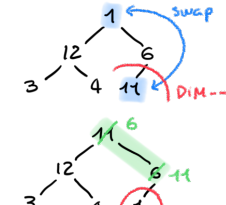
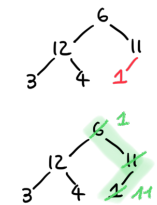
node : i
L-ch : $2*i$
R-ch : $2*i + 1$

CODE A PRIORITA'

Heap minimo

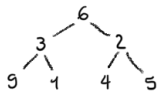
INSERIMENTO CON ORDINAMENTO

ESTRAZIONE → HEAP SORT (1)

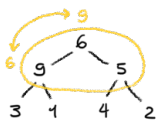
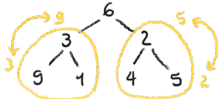


HEAPIFY
VETTORE → HEAP

Vett: 6 3 2 9 1 4 5
ROOT max

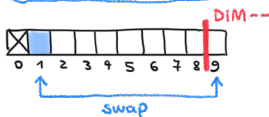


HEAPIFY



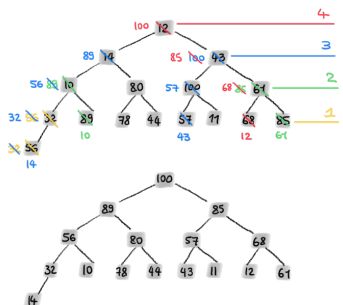
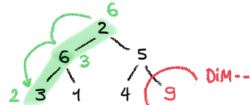
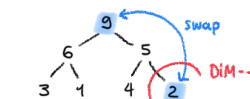
HEAP SORT

Estrazione da ROOT



Ordinamento fino a 0

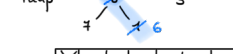
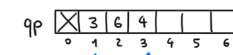
[REPEAT till DIM=0]



Inserire in seq. in CODA PRIORITA' di INDICI

SEQUENZA	PRIORITA'	ind	HASH
"si" = 10	3	0	ie
"insersca" = 15	6	1	ie
"in" = 5	4	2	si
"sequenza" = 18	7	3	in
"ie" = 4	1	4	inseri. seque.
		5	
		6	
		7	

HEAP → 4 CELLE



PROGRAMMAZIONE DINAMICA

PER PROBLEMI DI
OTTIMIZZAZIONE
CON
SOTTOSTRUTTURA
OPTIMA

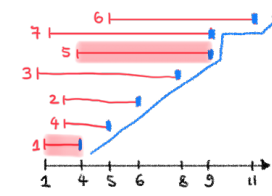
VS

PARADIGMA GREEDY

ALTERNATIVA
LOCALMENTE OPTIMA
FUNZIONE DI APPETIBILITÀ
⇓
SOL OPTIMA PER

NO MASSIMO DI ATTIVITÀ

A	inizio	fine
1	1	4
2	3	6
3	1	3
4	3	5
5	5	8
6	2	9
7	6	11



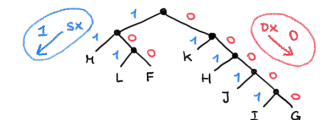
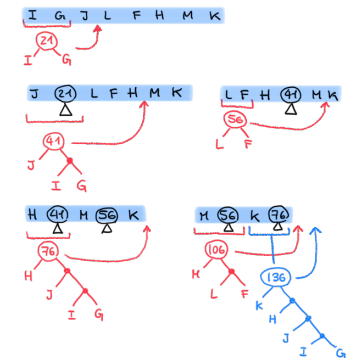
Sel la 1^a e quella via via compatibile

⇒ n° max 2 = P₁ P₅

CODICI DI HUFFMAN

- PREFIX-FREE
- CODICI → a SIMBOLI ⊕ FREQ.

G: 43 F: 29 H: 35 I: 8 L: 27 M: 50
J: 20 K: 60



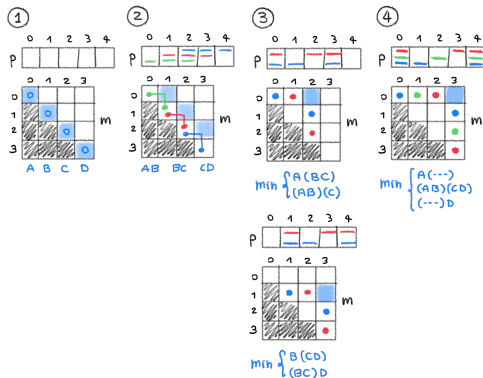
K = 01 H = 11 H = 001 F = 100
L = 101 J = 0001 G = 0000 I = 00001

PARENTESIZZAZIONE OPTIMA
DEL PRODOTTO DI MATRICI

OB: MIN n° di *

PARENT. POSSIBILI

- 1 ((A·B)·C)·D
- 2 (A·(B·C))·D
- 3 A ((B·C)·D)
- 4 A·(B·(C·D))
- 5 ((A·B)·(C·D))



Matrici A, B, C, D di dim
(4×4)(4×6)(6×15)(15×10)

n	0	1	2	3	4
p	4	4	6	15	10

Iterazione 1:
m[0,0] = ... = m[3,3] = 0

Iterazione 2:

AB m[0,1] = m[0,0] + m[1,1] + p[0]p[1]p[2] = 96

BC m[1,2] = m[1,1] + m[2,2] + p[1]p[2]p[3] = 360

CD m[2,3] = m[2,2] + m[3,3] + p[2]p[3]p[4] = 900

Iterazione 3:

m[0,2] = { m[0,0] + m[1,2] + p[0]p[1]p[3] = 600
m[0,1] + m[2,2] + p[0]p[2]p[3] = 456
(A·B) × C → m[0,2] min

m[1,3] = { m[1,1] + m[2,3] + p[1]p[2]p[4] = 1140
m[1,2] + m[3,3] + p[1]p[3]p[4] = 960
(B·C) × D → m[1,3] min

Iterazione 4:

m[0,3] = m[0,0] + m[1,3] + p[0]p[1]p[4] = 1120

m[0,1] + m[2,3] + p[0]p[2]p[4] = 1276

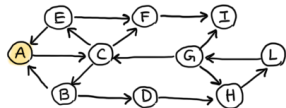
m[0,2] + m[3,3] + p[0]p[3]p[4] = 1056
(A·B) × C × D → m[0,3] min

⇒ ((A·B)·C)·D.

BFS

VISITA IN AMPIEZZA - BFS

Solo vertici raggiungibili da START



ORDINE ALFABETICO!

dist:	0	1	2	3	4	5	6
Q:	{A}	{C}	{B, E, F}	{D, I}	{H}	{L}	{G}
OUTPUT:	A	C	B, E, F	D, I	H	L	G



VISITE DEI GRAFI

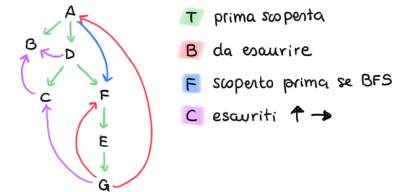
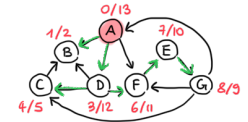
DFS

VISITA IN PROFONDITA' - DFS

visita TUTTI i vertici



tempo di scoperta / tempo fine elabor.



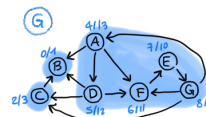
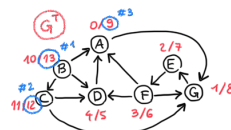
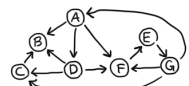
implementa

COMPONENTI FORTEMENTE CONNESSE - KOSARAJU

"cosa raggiungi?"

sotto G con V MUTUALMENTE RAGGIUNGIBILI

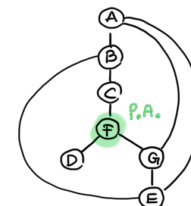
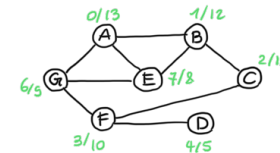
1. DFS (G^T)
 2. DFS (G) partendo da t_f
- $\Rightarrow$ $t_s = t_f$ S.C.C.



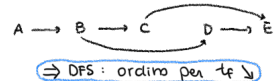
PUNTI DI ARTICOLAZIONE



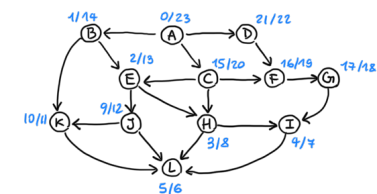
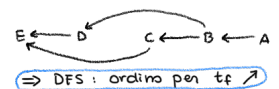
$\Rightarrow$ CHECK the DFS's TREE



ORDINAMENTO TOPOLOGICO dei DAG: 'DIRECTED ACYCLIC GRAPH'



INVERSO



OT: A D C F G B E J K H I L
OTI: L I H K J E B G F C D A

ON-LINE CONNECTIVITY

GRAFO NON ESPlicito

↓
insieme di vertici
e connessioni

id

0	1	2	3	4	5	6
---	---	---	---	---	---	---

(p-q)

● = ●

QUICKFIND

FIND → $O(1)$
UNION → $O(n)$

for (i=0...N)
id[i] = ●
↳ id[i] = ●

sequenza: 3-2 3-4 5-1 7-3 5-7 9-1
vertici: 0...9

for	id	0	1	2	3	4	5	6	7	8	9	
for	id	0	1	2	2	4	5	6	7	8	9	3-2
for	id	0	1	2	2	4	5	6	7	8	9	3-4
for	id	0	1	4	4	4	5	6	7	8	9	5-1
for	id	0	1	4	4	4	1	6	7	8	9	7-3
for	id	0	1	4	4	4	1	6	4	8	9	5-7
for	id	0	4	4	4	4	4	6	4	8	9	9-1
	id	0	4	4	4	4	4	6	4	8	9	



● = ●

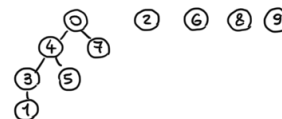
QUICKUNION

FIND → $O(n)$
UNION → $O(1)$

id[id[...●]] = id[id[...●]]

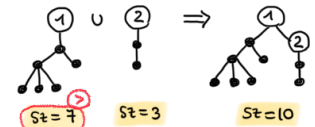
Seq: 1-3 3-4 5-1 4-0 7-3 5-7
vertici: 0...9

	id	0	1	2	3	4	5	6	7	8	9	
	id	0	1	2	3	4	5	6	7	8	9	1-3
	id	0	3	2	3	4	5	6	7	8	9	3-4
	id	0	3	2	4	4	5	6	7	8	9	5-1
	id	0	3	2	4	4	4	6	7	8	9	4-0
	id	0	3	2	4	4	4	6	7	8	9	7-3
	id	0	3	2	4	4	4	6	0	8	9	5-7
	id	0	3	2	4	4	4	6	0	8	9	



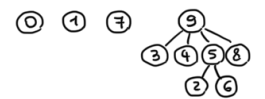
size
● = ●
● = ●

WEIGHTED QUICKUNION



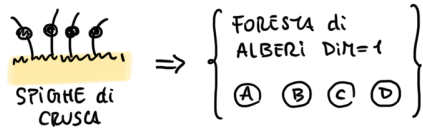
Seq: 3-9 4-3 2-5 6-2 3-8 3-5
vertici: 0...10

	id	0	1	2	3	4	5	6	7	8	9	10	
	st	1	1	1	1	1	1	1	1	1	1	1	3-9
	id	0	1	2	3	4	5	6	7	8	9	10	4-3
	st	1	1	1	1	1	1	1	1	1	2	1	4-3
	id	0	1	2	9	9	5	6	7	8	9	10	2-5
	st	1	1	1	1	1	1	1	1	1	3	1	2-5
	id	0	1	2	9	9	5	6	7	8	9	10	6-2
	st	1	1	1	1	1	2	1	1	1	3	1	6-2
	id	0	1	5	9	9	5	5	7	8	9	10	3-8
	st	1	1	1	1	1	3	1	1	1	4	1	3-8
	id	0	1	5	9	9	5	5	7	9	9	10	3-5
	st	1	1	1	1	1	3	1	1	1	7	1	3-5

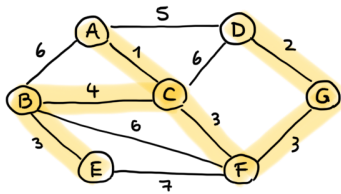


"MST" ALBERI RIVOLVENTI MINIMI

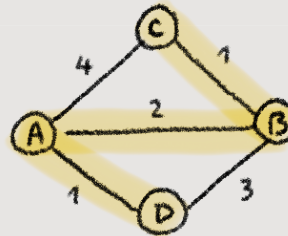
ALGORITMO di KRUSKAL



ARCHI in ordine PESO $\nearrow$
t.c.
ACICLICO



ordine: A-C, D-G, B-E,
C-F, F-G, B-C



ACICLICO con PESO MINIMO

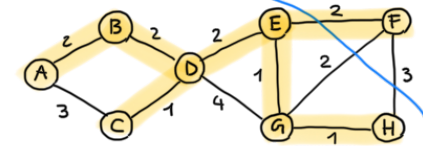
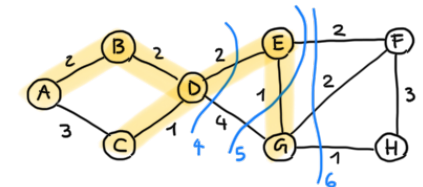
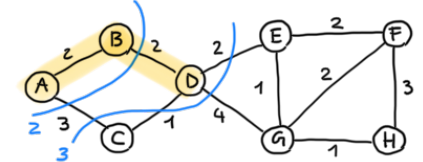
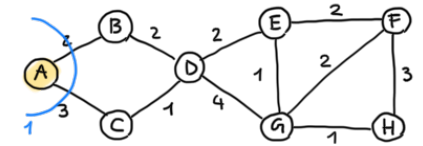
ALGORITMO di PRIM

$\{ \text{ALBERO} \}$
DIM=1

$V - W$

ARCO ADJ di PESO MIN

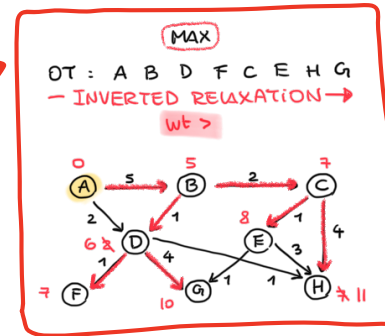
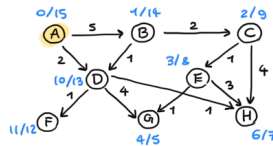
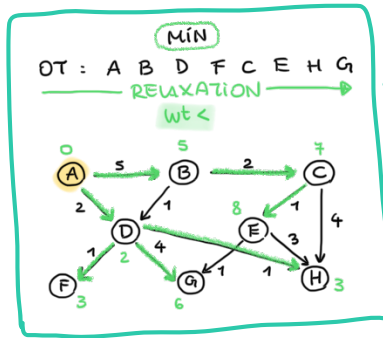
W t.c. $\notin$ $\{ \text{ALBERO} \}$



CAMMINI MINIMI E MASSIMI SU DAG PESATO

DFS $\rightarrow$
ORDINAMENTO TOPOLOGICO

MINIMI RELAXATION O.T.
MASSIMI INVERTED RELAXATION O.T.



DAG

CAMMINI MINIMI

wt

RIASSAMENTO di A

STIMA A STIMA B

$A \xrightarrow{wt} B$

$ST_A + wt < ST_B$

STIMA A $ST_A + wt$

$A \xrightarrow{wt} B$

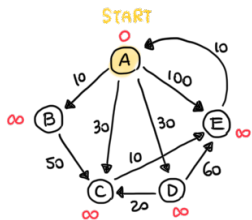
ALGORITMO DI DIJKSTRA

STRATEGIA GREEDY

-3
NOT NOT



PRIORITY-QUEUE $\{ \dots \}$
 $p = \text{min-dist}$
 $\rightarrow$ DEQUEUE & RELAX



PQ $\{A\}$: RELAX A

PQ $\{B, C, D, E\}$: RELAX B

PQ $\{C, D, E\}$: RELAX C

PQ $\{D, E\}$: RELAX D

PQ $\{E\}$: RELAX E

min-dist. from A:

A-0 B-10 C-30 D-30 E-40

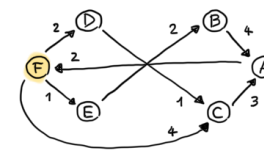
ALGORITMO DI BELLMAN-FORD

OK -3 OK

-3 -2
 4
wt = -1

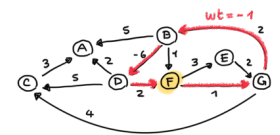


DETECTION



	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆
A	∞	7	6	-	-	-
B	∞	∞	3	3	-	-
C	∞	4	3	3	-	-
D	∞	2	2	2	-	-
E	∞	1	1	1	-	-
F	∞	0	0	0	-	-

REL all VISITED A...Z ORDER
CHECK NO VARIATION



	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆
A	∞	∞	∞	-1	-1	-1
B	∞	∞	3	2	-1	-1
C	∞	∞	5	2	1	-1
D	∞	∞	∞	-3	-4	-5
E	∞	3	3	2	1	-1
F	∞	0	0	-1	-2	-3
G	∞	1	1	0	-1	-2

REL all VISITED A...Z ORDER
FAILED CHECK

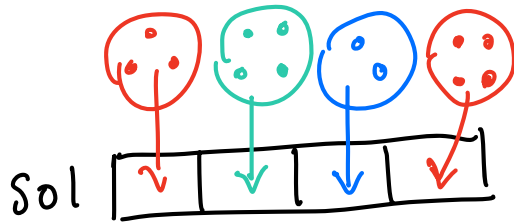
COMB SIMPLICI

no ORDINE

```
for (int i = start; ... )  
    sol → wrd[pos] = i;  
    ...  
    Comb Semp ( ... , i+1, pos+1 );
```

Princip. di MOLZIM

menu



```
int j = 0;  
while (menu[pos][j] != Null) {  
    | sol → port[pos] = strdup(menu[pos][j]);  
    | PrincipMolt(menu, sol, pos+1);  
    | }  
    return
```

SIMPLE PATH GRAFO

madj

sol $\rightarrow$ wrr[pos] = src;

visited[src] = 1;

//COND TER: DEST

if(src == dest){

↓

return;

}

//PASSO RECOR

for (int i = 0; i < G $\rightarrow$ V; i++) {

if (G $\rightarrow$ madj[src][i] != 0) { // EDGE

if (!visited[i]) { // not visited w

↓

SimpPath (G, sol, visited, ^{src}i, wt+1, pos + 1);

}

}

}

//COND TERM: FINITO

if (... > ...) {

...

}

//BACKTRACKING

visited[src] = 0;
